ปฏิบัติการสัปดาห์ที่ 12: เพิ่ม Advanced Validation และ Rate Limiter ให้ครบทุก Endpoint

วัตถุประสงค์ของปฏิบัติการ

เมื่อทำปฏิบัติการนี้เสร็จสิ้น นิสิตจะสามารถ

1. เขียน Zod schema ที่มี cross-field validation และ conditional validation สำหรับ endpoint ที่มีอยู่แล้วในโปรเจกต์ `student-api`
2. ติดตั้งและตั้งค่า `express-rate-limit` แยกโควตาตามความเสี่ยงของแต่ละ endpoint
3. ทดสอบ endpoint ที่ปรับปรุงแล้วด้วย Postman Collection Runner เพื่อยืนยันว่า rate limiter ทำงานถูกต้อง

ปฏิบัติการนี้ต่อยอดจากเนื้อหาบรรยายในเอกสาร `wk12.md` และเป็นใบงานส่งสัปดาห์ที่ 12 ตามที่กำหนดไว้ใน `course_plan.md` (วัดผล CLO1/PLO1) เนื้อหาของสัปดาห์นี้ยังเตรียมความพร้อมสำหรับสอบปฏิบัติ CLO4/PLO5 ในสัปดาห์ที่ 16 (Final) ซึ่งวัดทักษะการทดสอบและแก้ปัญหา Validation แบบสะสมจากสัปดาห์นี้ร่วมกับสัปดาห์ที่ 9 การสแกนด้วย OWASP ZAP ยังคงสอนไว้ในเนื้อหาบรรยาย (หัวข้อ 4.2 ของ `wk12.md`) แต่ย้ายมาเป็นแบบฝึกหัดต่อยอดในปฏิบัติการนี้ เนื่องจากคะแนนของใบงานสัปดาห์นี้ (1 คะแนน) ไม่สอดคล้องกับภาระงานหากบังคับให้ทำครบทั้ง Postman และ ZAP พร้อมเขียนรายงานแยก

สิ่งที่ต้องส่ง: สัปดาห์นี้ไม่มีไฟล์ใบงานแยกต่างหาก การส่งงานคือการทำโปรเจกต์ `student-api` ให้ผ่านครบตาม **จุดตรวจสอบที่ 1-3** ด้านล่าง (validation ครอบคลุม cross-field/conditional, rate limiter แยกโควตาถูกต้อง, ทดสอบผ่าน Postman) ผู้สอนตรวจจากโค้ดและผลการรันจริงในคาบปฏิบัติการ ไม่ต้องสร้างไฟล์รายงานเพิ่ม (ต่างจากสัปดาห์ที่ 13 ที่ต้องส่งไฟล์ `microservices-design.md` แยกต่างหาก)

เวลาที่ใช้

ช่วงเวลา	กิจกรรม
0-15 นาที	ทบทวนเนื้อหาสัปดาห์ที่ 11 (Review W11 — External API Integration & Resilience)
15-90 นาที	เพิ่ม Validation + Rate limiter ทุก endpoint
90-120 นาที	ทดสอบ Attack scenario ด้วย Postman

เครื่องมือที่ต้องใช้

1. โปรเจกต์ `student-api` จากปฏิบัติการสัปดาห์ที่ 1-11
2. Node.js และ npm
3. ไลบรารี `zod`, `express-rate-limit`
4. Postman (Collection Runner)
5. OWASP ZAP (สำหรับแบบฝึกหัดต่อยอดเท่านั้น ติดตั้งจาก https://www.zaproxy.org)

ส่วนที่ 1: ทบทวนเนื้อหาสัปดาห์ที่ 11 (0-15 นาที)

ขั้นตอนที่ 1.1 ตรวจสอบโปรเจกต์เดิม

```
cd student-api
npm test
npm run dev
```

ยืนยันว่าชุดทดสอบเดิมทั้งหมดผ่าน และ endpoint หลัก (`/api/v1/auth/login` , `/api/v1/students`) ยังทำงานได้ปกติ ก่อนเริ่มแก้ไข schema และเพิ่ม middleware ใหม่

ขั้นตอนที่ 1.2 สร้างโฟลเดอร์และไฟล์ validation schema

โปรเจกต์ `student-api` จากสัปดาห์ที่ 1-11 ยังไม่มีไฟล์ validation schema แยกเฉพาะ ให้สร้างโฟลเดอร์และไฟล์เปล่าไว้ก่อน จะเขียน schema จริงในส่วนที่ 2

```
mkdir -p schemas
touch schemas/studentSchema.js
```

จุดตรวจสอบที่ 1

- ชุดทดสอบเดิมรันผ่านทั้งหมดก่อนเริ่มแก้ไข
- สร้างไฟล์ `schemas/studentSchema.js` ไว้พร้อมสำหรับเขียน schema ในส่วนที่ 2

ส่วนที่ 2: เพิ่ม Validation และ Rate Limiter (15-90 นาที)

ขั้นตอนที่ 2.1 ติดตั้งไลบรารีที่จำเป็น

```
npm install express-rate-limit zod
```

ขั้นตอนที่ 2.2 เพิ่ม Conditional Validation ให้ schema การลงทะเบียน

เอกสารบรรยาย `wk12.md` หัวข้อ 2.3 สอนหลักการ **conditional validation** (ตรวจสอบ field หนึ่งแบบมีเงื่อนไขตามค่าของอีก field) โดยใช้ตัวอย่าง `studentType` / `passportNumber` / `nationalId` แต่ตาราง `students` จริงในโปรเจกต์ `student-api` (สร้างไว้ตั้งแต่สัปดาห์ที่ 3-5) มีแค่ field `name`, `major`, `email` **ไม่มี** `studentType` ดังนั้น **ห้าม copy ตัวอย่างในหัวข้อ 2.3 มาใช้ตรง ๆ** ให้เขียน schema ตามแนวคิดเดียวกัน (conditional validation ด้วย `.superRefine()`) แต่ปรับเงื่อนไขให้ใช้ field `major` ที่มีอยู่จริงแทน

เงื่อนไขของแบบฝึกหัดนี้: ถ้า `major` เริ่มต้นด้วยคำว่า `"International"` (เช่น `"International Business"`) ถือว่าเป็นสาขานานาชาติ ต้องแนบ `passportNumber` มาด้วยเสมอ ส่วนสาขาอื่นไม่ต้องแนบ

```javascript
// schemas/studentSchema.js
const { z } = require("zod");

const createStudentSchema = z
  .object({
    name: z.string().min(2).max(100),
    major: z.string().min(1).max(100),
    email: z.email(),
    passportNumber: z.string().optional(),
  })
  .superRefine((data, ctx) => {
    if (data.major.startsWith("International") && !data.passportNumber) {
      ctx.addIssue({
        code: "custom",
        message: "ต้องระบุ passportNumber สำหรับสาขานานาชาติ",
        path: ["passportNumber"],
      });
    }
  });

module.exports = { createStudentSchema };
```

หมายเหตุ: `passportNumber` ใช้สำหรับ validate ข้อมูลที่ส่งเข้ามาเท่านั้น ตาราง `students` ไม่มีคอลัมน์นี้ จึง **ไม่ต้อง** นำ `passportNumber` ไปใส่ในคำสั่ง `INSERT` ที่ขั้นตอน 2.3 เป้าหมายของแบบฝึกหัดนี้คือฝึกเขียน conditional validation ไม่ใช่เพิ่ม field ใหม่ในฐานข้อมูล

ขั้นตอนที่ 2.3 นำ Schema ไปใช้ใน Route `POST /api/v1/students`

เปิดไฟล์ route ที่มี handler ของ `POST /api/v1/students` อยู่แล้ว (`routes/students.js` ที่สร้างไว้ตั้งแต่สัปดาห์ที่ 3-4) เพิ่มการ import `createStudentSchema` ที่หัวไฟล์ แล้วแทรกโค้ด validate ไว้เป็นบรรทัดแรก

ในฟังก์ชัน handler เดิม ก่อนส่วนที่บันทึกข้อมูลลงฐานข้อมูล **route เดิมของนิสิตมี middleware `createStudentLimiter` อยู่แล้ว (rate limit จากสัปดาห์ที่ 4) และไม่มี `authenticateToken` — ให้คงไว้ตามเดิม อย่าลบหรือเพิ่มเติม** แค่แทรกโค้ด validate เข้าไปในฟังก์ชัน handler เท่านั้น

```javascript
// routes/students.js
const { createStudentSchema } = require("../schemas/studentSchema"); // เพิ่มบรรทัดนี้ที่หัวไฟล์

router.post("/", createStudentLimiter, async (req, res, next) => {
  // --- เพิ่มโค้ดนี้ไว้บรรทัดแรกของ handler เดิม (แทนที่ validateStudentBody เดิม) ---
  const result = createStudentSchema.safeParse(req.body);
  if (!result.success) {
    return res.status(400).json({
      error: {
        code: "VALIDATION_ERROR",
        message: "ข้อมูลที่ส่งมาไม่ถูกต้อง",
        details: result.error.issues.map((issue) => ({
          field: issue.path.join("."),
          message: issue.message,
        })),
      },
    });
  }
  // --- จบส่วนที่เพิ่มใหม่ ---

  // ส่วนที่เหลือคือโค้ด handler เดิมของสัปดาห์ที่ 3-4 (เช่น INSERT ลงตาราง students)
  // ให้เปลี่ยนจากใช้ req.body เป็น result.data ที่ผ่านการตรวจสอบแล้วแทน
  // ตัวอย่าง: const { name, major, email } = result.data;
  // (ไม่ต้องดึง passportNumber ออกมา เพราะไม่มีคอลัมน์นี้ใน INSERT)
});
```

ขั้นตอนที่ 2.4 ทดสอบ Schema ใหม่ด้วย Postman

ส่ง `POST /api/v1/students` พร้อม body

```json
{
  "name": "John Smith",
  "email": "john@example.com",
  "major": "International Business"
}
```

(ไม่แนบ `passportNumber`) ผลลัพธ์ที่คาดหวังคือ `400 Bad Request` พร้อม field `details` ระบุว่า `passportNumber` เป็นปัญหา

```json
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "ข้อมูลที่ส่งมาไม่ถูกต้อง",
    "details": [
      { "field": "passportNumber", "message": "ต้องระบุ passportNumber สำหรับสาขานานาชาติ"
}
    ]
  }
}
```

ขั้นตอนที่ 2.5 สร้าง Rate Limiter แยกตามความเสี่ยง

```javascript
// middlewares/rateLimiter.js
const rateLimit = require("express-rate-limit");

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  standardHeaders: "draft-7",
  legacyHeaders: false,
  message: {
    error: {
      code: "TOO_MANY_REQUESTS",
      message: "พยายามเข้าสู่ระบบเกินจำนวนที่กำหนด กรุณาลองใหม่ภายหลัง",
    },
  },
});

const generalLimiter = rateLimit({
  windowMs: 60 * 1000,
  max: 100,
  standardHeaders: "draft-7",
  legacyHeaders: false,
});

module.exports = { authLimiter, generalLimiter };
```

ขั้นตอนที่ 2.6 นำ Rate Limiter ไปใช้ใน app.js

โปรเจกต์ `student-api` แยก `app.js` (ตั้งค่า middleware และ route) ออกจาก `server.js` (เรียก `app.listen()`) มาตั้งแต่สัปดาห์ที่ 9 ให้แก้ไขที่ `app.js` — **ไม่ใช่** `server.js` — เพราะ rate limiter ต้องทำงานก่อน route ทุกตัวที่อยู่ใน `app.js`

เส้นทาง login จริงของโปรเจกต์นี้คือ `/api/v1/auth/login` (ไม่ใช่ `/auth/login`) เพราะ `authRouter` ถูก mount ไว้ที่ `/api/v1/auth` ให้ยึดตาม path จริงในไฟล์ `app.js` ของตนเองเป็นหลัก

```javascript
// app.js
const express = require("express");
const app = express();

app.use(express.json()); // บรรทัดเดิม

// --- เพิ่มโค้ด 3 บรรทัดนี้ ต่อจาก app.use(express.json()) และก่อน route ทั้งหมด ---
const { authLimiter, generalLimiter } = require("./middlewares/rateLimiter");
app.set("trust proxy", 1);
app.use(generalLimiter);
// --- จบส่วนที่เพิ่มใหม่ ---

// --- เพิ่ม authLimiter ก่อน mount authRouter เสมอ (แทรกก่อนบรรทัด
app.use("/api/v1/auth", authRouter)) ---
app.use("/api/v1/auth/login", authLimiter);
app.use("/api/v1/auth", authRouter);

// ... route อื่น ๆ ที่เหลือจากสัปดาห์ก่อนหน้า ...

module.exports = app;
```

ตำแหน่งของ `app.use(generalLimiter)` ต้องอยู่หลัง `express.json()` แต่ก่อน route ทั้งหมด เพื่อให้ทุก request ถูกนับก่อนเข้าสู่ business logic ส่วน `app.use("/api/v1/auth/login", authLimiter)` ต้องอยู่ **ก่อน** บรรทัดที่ mount `authRouter` เพื่อให้ middleware นี้ทำงานก่อนเข้า handler ของ `/login`

ขั้นตอนที่ 2.7 รันชุดทดสอบเดิมอีกครั้ง

```
npm test
```

หากชุดทดสอบ Jest/Supertest เดิมยิง `POST /api/v1/auth/login` ซ้ำหลายครั้งในไฟล์เดียวกัน อาจถูก rate limiter บล็อกจนทดสอบไม่ผ่าน ให้ปรับ `windowMs` / `max` เฉพาะใน `NODE_ENV=test` ให้สูงขึ้นชั่วคราว

```javascript
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: process.env.NODE_ENV === "test" ? 1000 : 5,
  standardHeaders: "draft-7",
  legacyHeaders: false,
});
```

จุดตรวจสอบที่ 2

- `POST /api/v1/students` ปฏิเสธ request ที่ขาด `passportNumber` ตามเงื่อนไข `major`
- `generalLimiter` และ `authLimiter` ทำงานแยกโควตากันตามที่ตั้งค่า
- ชุดทดสอบ Jest/Supertest เดิมยังรันผ่านทั้งหมดหลังเพิ่ม rate limiter

---

ส่วนที่ 3: ทดสอบ Attack Scenario ด้วย Postman (90-120 นาที)

ขั้นตอนที่ 3.1 จำลอง Brute-force ด้วย Postman Collection Runner

เปิด Collection ของ `student-api` เลือก request `POST /api/v1/auth/login` แล้วเปิด Collection Runner ตั้งค่า

ตัวเลือก	ค่าที่ตั้ง
Iterations	20
Delay	0 ms

กด Run แล้วสังเกตผลลัพธ์ ควรพบว่า request ที่ 1-5 ได้รับ `200` หรือ `401` ตามความถูกต้องของรหัสผ่าน ส่วนตั้งแต่ request ที่ 6 เป็นต้นไปได้รับ `429 Too Many Requests` ทั้งหมด

ขั้นตอนที่ 3.2 ตรวจสอบ Header การตอบกลับ

ตรวจสอบ header ของ response ที่ได้ `429` ต้องมี header ชื่อ `RateLimit` ที่ระบุค่า `limit`, `remaining=0` และ `reset` รวมอยู่ในค่าเดียวกัน ตามมาตรฐาน IETF draft-7 ที่ `standardHeaders: "draft-7"` เปิดใช้งาน

จุดตรวจสอบที่ 3

- Postman Collection Runner แสดงผล `429` ตั้งแต่ request ที่เกินโควตาของ `authLimiter`
- Header ของ response `429` มีค่า `RateLimit` ครบตามที่ตั้งค่า

ส่วนต่อยอด: แบบฝึกหัดเพิ่มเติมด้วยตนเอง

แบบฝึกหัดที่ 1: Cross-field Validation เพิ่มเติม

เพิ่ม schema สำหรับ endpoint การลงทะเบียนเรียน (หรือ endpoint ที่เทียบเท่าในโครงงานของตนเอง) ที่ตรวจสอบว่า `maxCredits` ต้องไม่เกิน 22 หน่วยกิต เมื่อ `studentStatus` เป็น `"probation"` แต่ไม่เกิน 25 หน่วยกิตในกรณีปกติ โดยใช้ `.superRefine()`

แบบฝึกหัดที่ 2: Rate Limiter แยกตาม Role

สร้าง rate limiter ใหม่ชื่อ `userLimiter` (ต่อยอดจาก pattern ในหัวข้อ 3.4 ของ `wk12.md` ที่ใช้ `keyGenerator` แยกตาม `req.user.id` และ `ipKeyGenerator(req.ip)` เป็น fallback) แล้วปรับให้จำกัดโควตาต่างกันตาม `req.user.role` เช่น `admin` ได้โควตา 200 ครั้งต่อนาที ส่วน `student` ได้ 30 ครั้งต่อนาที โดยใช้ `keyGenerator` และ `max` แบบฟังก์ชัน (ดูตัวอย่างการตั้งค่า `max` เป็นฟังก์ชันตาม role ได้ที่หัวข้อ 4.1 ของ ref-express-rate-limit.md — ไม่มีสอนในเนื้อหาบรรยาย wk12.md) จากนั้นนำ `userLimiter` ไปใช้กับ route ที่ต้องยืนยันตัวตนแล้ว (เช่น `/api/v1/students`) ต่อจาก `authenticateToken` เสมอ

แบบฝึกหัดที่ 3: ทดสอบ Injection พื้นฐาน

ลองส่ง payload ที่มีเครื่องหมาย SQL พิเศษ เช่น `{ "name": "'; DROP TABLE students; --" }` ไปยัง `POST /api/v1/students` แล้วตรวจสอบว่าระบบยังปลอดภัยเนื่องจากใช้ parameterized query ตามสัปดาห์ที่ 5

แบบฝึกหัดที่ 4: สแกนหาช่องโหว่ด้วย OWASP ZAP

OWASP ZAP เป็นโปรแกรม (ไม่ใช่บริการออนไลน์หรือ package ของ Node.js) ติดตั้งได้ 2 วิธีจาก https://www.zaproxy.org/download/

- **แบบโปรแกรม GUI (แนะนำสำหรับผู้เริ่มต้น)**: ดาวน์โหลดตัวติดตั้งตามระบบปฏิบัติการของเครื่อง (มีให้ทั้ง Windows/Mac/Linux) แล้วติดตั้งเหมือนโปรแกรมทั่วไป เปิดใช้งานเป็นหน้าต่าง GUI ของตัวเอง แยกจากเครื่องมือที่ใช้มาตลอดวิชานี้อย่าง Postman/VS Code
- **แบบ Docker (สำหรับผู้ที่คุ้นเคยกับ Docker อยู่แล้ว)**: ใช้คำสั่ง `docker run -t ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t <URL>` แทนการติดตั้งโปรแกรม แต่มีข้อควรระวังสำคัญคือ **ต้องใช้** `http://host.docker.internal:3000` **แทน** `http://localhost:3000` เพราะ container มองไม่เห็น `localhost` ของเครื่องที่รัน Docker โดยตรง (Docker Desktop บน Windows/Mac รองรับ `host.docker.internal` ให้อัตโนมัติ แต่ Linux อาจต้องตั้งค่าเพิ่มเติม) หากยังไม่คุ้นเคยกับ Docker หรือเจอปัญหาเชื่อมต่อ ให้ใช้วิธี GUI ด้านบนแทน

ก่อนเปิด ZAP ต้องรัน `student-api` **ให้ทำงานอยู่ก่อนเสมอ** เพราะ ZAP สแกนระบบที่กำลังทำงานจริงผ่าน HTTP ไม่ใช่สแกนไฟล์โค้ดในเครื่อง

```
npm run dev
```

ยืนยันว่าเปิด `http://localhost:3000` ในเบราว์เซอร์แล้วเห็นระบบทำงานปกติก่อน จึงเปิด ZAP ตามหลักการในหัวข้อ 4.2 ของ `wk12.md` แล้วดำเนินการ

> **หมายเหตุ**: ZAP เป็นโปรแกรมที่เขียนด้วย Java ใช้ RAM ค่อนข้างมากระหว่างสแกน หากเครื่องช้าควรปิดโปรแกรมอื่นที่ไม่จำเป็นก่อน และแบบฝึกหัดนี้เป็นส่วนต่อยอด (ไม่บังคับ) ตามที่ระบุไว้ในหัวข้อ "เครื่องมือที่ต้องใช้" ข้อ 5

1. เปิด OWASP ZAP เลือก **Automated Scan** ระบุ URL เป็น `http://localhost:3000`
2. กด **Attack** แล้วรอจนสแกนเสร็จ (Spider ตามด้วย Active Scan)
3. เปิดแท็บ **Alerts** ดูรายการช่องโหว่ที่พบ เรียงตามระดับความเสี่ยง
4. สร้างไฟล์ `security-scan-report.md` ที่ระดับรากของโปรเจกต์ `student-api` บันทึกผลตามรูปแบบ

```markdown
# รายงานผลการสแกนความปลอดภัยด้วย OWASP ZAP

| ระดับความเสี่ยง | ช่องโหว่ที่พบ | Endpoint | ข้อเสนอแนะการแก้ไข |
| --- | --- | --- | --- |
| ... | ... | ... | ... |
```

หากพบช่องโหว่ระดับ Medium ขึ้นไป ให้แก้ไขทันทีในโปรเจกต์และบันทึกในรายงานว่าได้แก้ไขแล้วหรือยัง ต่อยอดเพิ่มเติมได้ด้วยการตั้งค่า ZAP ให้ล็อกอินก่อนสแกน (Authenticated Scan) เพื่อตรวจสอบ endpoint ที่ต้องมี token ได้ครอบคลุมกว่าการสแกนแบบไม่ล็อกอิน

แบบฝึกหัดที่ 5: เขียน Test อัตโนมัติสำหรับ Rate Limiter

เขียน Jest/Supertest test case ที่ยิง `POST /api/v1/auth/login` ซ้ำเกินโควตาแล้วยืนยันว่าได้รับ `429` ในการเรียกครั้งที่เกิน โดยต้องแยก environment ของ test นี้ออกจาก test อื่นเพื่อไม่ให้ rate limiter ในโหมด test รบกวนกัน

เกณฑ์การประเมินแบบฝึกหัดต่อยอด

รายการ	ผ่านเกณฑ์เมื่อ
แบบฝึกหัดที่ 1	Schema ปฏิเสธจำนวนหน่วยกิตที่เกินเงื่อนไขตาม `studentStatus` ได้ถูกต้องทั้งสองกรณี
แบบฝึกหัดที่ 2	ผู้ใช้ role ต่างกันได้โควตา rate limit ต่างกันตามที่กำหนด
แบบฝึกหัดที่ 3	ระบบไม่เกิดข้อผิดพลาดที่เผยโครงสร้างฐานข้อมูล และข้อมูลไม่ถูกเปลี่ยนแปลงผิดปกติ
แบบฝึกหัดที่ 4	ไฟล์ `security-scan-report.md` มีผลการสแกนอย่างน้อย 1 รอบ พร้อมข้อเสนอแนะการแก้ไขที่นำไปปฏิบัติได้จริง

รายการ	ผ่านเกณฑ์เมื่อ
แบบฝึกหัดที่ 5	Test case รันผ่านและยืนยัน `429` ได้โดยไม่กระทบชุดทดสอบอื่น

เนื้อหาสัปดาห์ถัดไป (สัปดาห์ที่ 13) จะขยายผลจากการป้องกันระดับ endpoint ในสัปดาห์นี้ไปสู่การออกแบบสถาปัตยกรรมระดับ Microservices ซึ่งต้องนำ validation และ rate limiter ที่ตั้งค่าไว้ในสัปดาห์นี้ไปใช้ทั้งที่ระดับ API Gateway และระดับบริการย่อย